

Convenções de Código em Activities

Professor: Henrique Galati
IFSP - Campus Araraquara
Análise e Desenvolvimento de Sistemas

A estrutura interna de uma classe **Activity** pode afetar significativamente a **leitura, manutenção, reuso e escalabilidade** do código. Adotar uma **ordem padrão e consistente** facilita a colaboração em equipe e torna mais fácil encontrar e entender partes específicas do código.

Relembrando: O que é uma **Activity**?

Uma **Activity** é um dos principais componentes de um app Android. Ela representa **uma única tela com interface de usuário**. Exemplo de Activities comuns:

- **MainActivity**: tela inicial
- **LoginActivity**: tela de login
- **DetailActivity**: exibe os detalhes de algo

Código que iremos refatorar

Para aplicarmos as convenções dentro de uma Activity, usaremos o exemplo que desenvolvemos na última aula:

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    android:background="?attr/background">
```

```

<LinearLayout
    android:id="@+id/cardCliente"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="?attr/colorPrimary"
    android:padding="16dp"
    android:orientation="horizontal">

    <ImageView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:src="@mipmap/ic_launcher_round">

    </ImageView>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:gravity="center"
        android:layout_weight="2"
        android:orientation="vertical">

        <TextView
            android:id="@+id/tvNome"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/nome_padrao"
            android:textColor="?attr/colorOnPrimary"
            android:textSize="18sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/tvEmail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/email_padrao"
            android:textColor="?attr/colorOnPrimary"
            android:textSize="16sp" />
    </LinearLayout>

</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="10dp"
    android:orientation="vertical">

    <EditText
        android:id="@+id/etNome"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="10dp"
        android:textColorHint="?attr/colorPrimary"
        android:hint="@string/hint_nome" />

```

```

<EditText
    android:id="@+id/etEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="10dp"
    android:textColorHint="?attr/colorPrimary"
    android:hint="@string/hint_email" />

<Button
    android:id="@+id/btnAlterar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_margin="10dp"
    android:backgroundTint="?attr/colorSecondary"
    android:textColor="?attr/colorOnSecondary"
    android:text="@string/alterar" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center">

    <Button
        android:id="@+id/btnDayNight"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_margin="10dp"
        android:backgroundTint="?attr/colorSecondary"
        android:textColor="?attr/colorOnSecondary"
        android:text="@string/daynight" />

    <Button
        android:id="@+id/btnIdioma"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_margin="10dp"
        android:backgroundTint="?attr/colorSecondary"
        android:textColor="?attr/colorOnSecondary"
        android:text="@string/idioma" />
</LinearLayout>
</LinearLayout>

```

MainActivity.kt

```

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

```

```

import androidx.appcompat.app.AppCompatActivity
import androidx.core.os.LocaleListCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnDayNight = findViewById<Button>(R.id.btnDayNight)
        val btnIdioma = findViewById<Button>(R.id.btnIdioma)
        val btnAlterar = findViewById<Button>(R.id.btnAlterar)
        val etNome = findViewById<EditText>(R.id.etNome)
        val etEmail = findViewById<EditText>(R.id.etEmail)
        val tvNome = findViewById<TextView>(R.id.tvNome)
        val tvEmail = findViewById<TextView>(R.id.tvEmail)

        btnDayNight.setOnClickListener {
            val isNight = AppCompatActivity.getDefaultNightMode() ==
                AppCompatActivity.MODE_NIGHT_YES

            val mode = if (isNight) {
                AppCompatActivity.MODE_NIGHT_NO
            } else {
                AppCompatActivity.MODE_NIGHT_YES
            }

            AppCompatActivity.setDefaultNightMode(mode)
        }

        btnIdioma.setOnClickListener {
            val idiomaAtual = AppCompatActivity.getApplicationLocales()
            val usandoIdiomaPadrao =
                !idiomaAtual.isEmpty() && idiomaAtual[0]?.language == "en"
            val proximoIdioma = if (usandoIdiomaPadrao) "pt" else "en"
            AppCompatActivity.setApplicationLocales(
                LocaleListCompat.forLanguageTags(proximoIdioma)
            )
        }

        btnAlterar.setOnClickListener {
            tvNome.text = etNome.text.toString()
            tvEmail.text = etEmail.text.toString()
        }
    }
}

```

res/values/colors.xml

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="primary">#6200EE</color>
    <color name="secondary">#03DAC5</color>
    <color name="background">#FFFFFF</color>
    <color name="onPrimary">#FFFFFF</color>

```

```
        <color name="onSecondary">#FFFFFF</color>
    </resources>
```

res/values/strings.xml

```
<resources>
    <string name="app_name">Perfil</string>
    <string name="nome_padrao">Client name</string>
    <string name="email_padrao">email@client.com</string>
    <string name="hint_nome">Type your name</string>
    <string name="hint_email">Type your email address</string>
    <string name="alterar">Change</string>
    <string name="daynight">Day / Night</string>
    <string name="idioma">English</string>
</resources>
```

res/values/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Base.Theme.Perfil" parent="Theme.Material3.DayNight.NoActionBar">
        <item name="colorPrimary">@color/primary</item>
        <item name="colorOnPrimary">@color/onPrimary</item>
        <item name="colorSecondary">@color/secondary</item>
        <item name="colorOnSecondary">@color/onSecondary</item>
        <item name="background">@color/background</item>
        <item name="android:windowBackground">@color/primary</item>
        <item name="android:textSize">18sp</item>
    </style>

    <style name="Theme.Perfil" parent="Base.Theme.Perfil" />
</resources>
```

res/values-night/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="primary">#CFBCFF</color>
    <color name="onPrimary">#381E72</color>
    <color name="background">#1C1B1F</color>
    <color name="secondary">#E6E1E5</color>
    <color name="onSecondary">#1C1B1F</color>
</resources>
```

res/values-night/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Base.Theme.Perfil" parent="Theme.Material3.DayNight.NoActionBar">
        <item name="colorPrimary">@color/primary</item>
        <item name="colorOnPrimary">@color/onPrimary</item>
    </style>
</resources>
```

```

        <item name="colorSecondary">@color/secondary</item>
        <item name="colorOnSecondary">@color/onSecondary</item>
        <item name="background">@color/background</item>
        <item name="android:windowBackground">@color/primary</item>
        <item name="android:textSize">18sp</item>
    </style>
</resources>

```

res/values-pt-rBR/strings.xml

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Perfil</string>
    <string name="nome_padrao">Nome do Cliente</string>
    <string name="email_padrao">email@cliente.com</string>
    <string name="hint_nome">Digite o nome</string>
    <string name="hint_email">Digite o email</string>
    <string name="alterar">Alterar</string>
    <string name="daynight">Dia / Noite</string>
    <string name="idioma">Português</string>
</resources>

```

Estrutura Recomendada de uma **Activity**

Abaixo está uma **ordem sugerida** para organizar os elementos dentro de uma **Activity** Kotlin moderna usando boas práticas:

- Constantes
- Atributos da Classe
- Métodos do ciclo de vida da Activity (onCreate e outros)
- Funções de Interface (quando aplicável)
- Funções Auxiliares (setup inicial de interface e listeners)
- Funções Utilitárias (regra de negócio)

1) Constantes

Definidas dentro do **companion object**. Um **companion object** é um **objeto único** associado a uma classe, que permite acessar **funções e variáveis diretamente pela classe**, sem precisar criar um objeto. Ele **substitui** o conceito de **métodos static** do Java. Utilizadas para **chaves, tags de Log, request codes**, entre outros.

Constante padrão:

```
companion object {  
    const val TAG = "MainActivity"  
}
```

2) Atributos da Classe

Até agora, somente utilizamos variáveis dentro do escopo de onCreate. Isso pode ser um problema, pois após executar o método, perdemos as referências dos objetos que declaramos dentro deles, embora os listeners funcionem. Por convenção, declaramos estes objetos como atributos para utilizá-los em outras funções. Organize os atributos em 3 grupos:

- **val**: valores imutáveis (ex.: `ViewModel`, argumentos, injeções).
- **var**: valores mutáveis (ex.: estados internos, contadores).
- **lateinit var**: para objetos não nulos que serão inicializados depois (ex.: `Button`, `EditText`). Em uma `Activity`, as Views só existem **depois** de `setContentView(...)`. Não dá para inicializá-las no construtor (o sistema cria a `Activity`). Por isso usamos `lateinit`. Na maioria dos casos, **val** e **var** exigem que você inicialize no momento em que é declarado.

3) Funções do ciclo de vida da Activity

A função mais comum e obrigatória do ciclo de vida da `Activity` é o `onCreate`. Ela é obrigatória e configura o funcionamento da `Activity`. Mais a frente na disciplina iremos trabalhar com outros métodos de ciclo de vida. Eles deverão vir logo após o método `onCreate`.

4) Funções de Interface (quando aplicável)

Se a `Activity` implementa interfaces (ex.: `View.OnClickListener`), coloque essas funções logo após o ciclo de vida.

5) Funções Auxiliares de Configuração

Agrupe funções auxiliares que ajudam a separar responsabilidades:

- `setupViews()` → configura visual da tela (geralmente os métodos `findViewById`).
- `setupListeners()` → define os eventos (geralmente dos botões).

6) Funções Utilitárias

são funções criadas dentro da `Activity` que **ajudam a organizar a lógica da aplicação**, mas que **não fazem parte diretamente do ciclo de vida**, nem da configuração da interface (como `setupViews` ou `setupListeners`). Eles **realizam tarefas específicas e reutilizáveis**, como:

- Atualizar um texto na tela.
- Validar campos.
- Alternar o idioma.
- Mostrar um `Toast`.
- Alternar entre modo claro/escuro.

MainActivity.kt (versão refatorada aplicando as boas práticas)

```
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatActivity
import androidx.core.os.LocaleListCompat

class MainActivity : AppCompatActivity() {

    companion object {
        const val TAG = "MainActivity"
    }

    private lateinit var btnDayNight: Button
    private lateinit var btnIdioma: Button
    private lateinit var btnAlterar: Button
    private lateinit var etNome: EditText
    private lateinit var etEmail: EditText
    private lateinit var tvNome: TextView
    private lateinit var tvEmail: TextView
```

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    setupViews()
    setupListeners()
}

private fun setupViews() {
    btnDayNight = findViewById(R.id.btnDayNight)
    btnIdioma = findViewById(R.id.btnIdioma)
    btnAlterar = findViewById(R.id.btnAlterar)
    etNome = findViewById(R.id.etNome)
    etEmail = findViewById(R.id.etEmail)
    tvNome = findViewById(R.id.tvNome)
    tvEmail = findViewById(R.id.tvEmail)
}

private fun setupListeners() {
    btnDayNight.setOnClickListener { mudarTema() }
    btnIdioma.setOnClickListener { alternarIdioma() }
    btnAlterar.setOnClickListener { atualizarTexto() }
}

/*
Versão alternativa, usando referência direta
private fun setupListeners() {
    btnDayNight.setOnClickListener(::mudarTema)
    btnIdioma.setOnClickListener(::alternarIdioma)
    btnAlterar.setOnClickListener(::atualizarTexto)
}
*/

private fun mudarTema() {
    val isNight = AppCompatActivity.getDefaultNightMode() ==
        AppCompatActivity.MODE_NIGHT_YES

    val mode = if (isNight) {
        AppCompatActivity.MODE_NIGHT_NO
    } else {
        AppCompatActivity.MODE_NIGHT_YES
    }

    AppCompatActivity.setDefaultNightMode(mode)
}

private fun alternarIdioma() {
    val idiomaAtual = AppCompatActivity.getApplicationLocales()
    val usandoIngles = !idiomaAtual.isEmpty() && idiomaAtual[0]?.language == "en"
    val proximoIdioma = if (usandoIngles) "pt" else "en"

    AppCompatActivity.setApplicationLocales(
        LocaleListCompat.forLanguageTags(proximoIdioma))
}

```

```

        private fun atualizarTexto() {
            tvNome.text = etNome.text.toString()
            tvEmail.text = etEmail.text.toString()
        }
    }
}

```

Veja abaixo uma versão alternativa da classe MainActivity em que utiliza uma interface que captura todos os cliques em uma View. Configuramos os botões para responderem de acordo.

MainActivity.kt (versão alternativa)

```

import android.os.Bundle
import android.view.View
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatActivity
import androidx.core.os.LocaleListCompat

class MainActivity : AppCompatActivity(), View.OnClickListener {

    companion object {
        const val TAG = "MainActivity"
    }

    private lateinit var btnDayNight: Button
    private lateinit var btnIdioma: Button
    private lateinit var btnAlterar: Button
    private lateinit var etNome: EditText
    private lateinit var etEmail: EditText
    private lateinit var tvNome: TextView
    private lateinit var tvEmail: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        setupViews()
        setupListeners()
    }

    override fun onClick(view: View?) {
        when (view?.id) {
            R.id.btnDayNight -> mudarTema()
            R.id.btnIdioma -> alternarIdioma()
            R.id.btnAlterar -> atualizarTexto()
        }
    }

    private fun setupViews() {
        btnDayNight = findViewById(R.id.btnDayNight)
    }
}

```

```

        btnIdioma = findViewById(R.id.btnIdioma)
        btnAlterar = findViewById(R.id.btnAlterar)
        etNome = findViewById(R.id.etNome)
        etEmail = findViewById(R.id.etEmail)
        tvNome = findViewById(R.id.tvNome)
        tvEmail = findViewById(R.id.tvEmail)
    }

    private fun setupListeners() {
        // Define a própria Activity como listener
        btnDayNight.setOnClickListener(this)
        btnIdioma.setOnClickListener(this)
        btnAlterar.setOnClickListener(this)
    }

    private fun mudarTema() {
        val isNight = AppCompatActivity.getDefaultNightMode() ==
            AppCompatActivity.MODE_NIGHT_YES

        val mode = if (isNight) {
            AppCompatActivity.MODE_NIGHT_NO
        } else {
            AppCompatActivity.MODE_NIGHT_YES
        }

        AppCompatActivity.setDefaultNightMode(mode)
    }

    private fun alternarIdioma() {
        val idiomaAtual = AppCompatActivity.getApplicationLocales()
        val usandoIngles = !idiomaAtual.isEmpty() && idiomaAtual[0]?.language == "en"
        val proximoIdioma = if (usandoIngles) "pt" else "en"

        AppCompatActivity.setApplicationLocales(
            LocaleListCompat.forLanguageTags(proximoIdioma)
        )
    }

    private fun atualizarTexto() {
        tvNome.text = etNome.text.toString()
        tvEmail.text = etEmail.text.toString()
    }
}

```

O que é o **Toast** no Android?

O **Toast** é uma forma simples e rápida de **mostrar mensagens temporárias na tela** para o usuário. Ele serve para exibir **avisos curtos e informativos**, sem exigir interação.

Exemplo: “Campos obrigatórios não preenchidos”, “Item salvo com sucesso”, “Erro ao conectar”.

Ele é **leve**, pois aparece como uma pequena caixa flutuante, geralmente na parte inferior da tela. É **temporário**, pois desaparece automaticamente após alguns segundos. Não **interrompe** o fluxo do aplicativo. **Não exige clique** ou qualquer ação do usuário.

Uso básico:

```
Toast.makeText(context, "Mensagem para o usuário", Toast.LENGTH_SHORT).show()
```

- `context`: normalmente é `this` ou `applicationContext`.
- `"Mensagem para o usuário"`: é o texto exibido.
- `Toast.LENGTH_SHORT` ou `Toast.LENGTH_LONG`: define a duração da exibição.
- `.show()`: exibe o Toast.

Exemplo em botão (não usando as boas práticas por questões didáticas):

```
btnSalvar.setOnClickListener {  
    val nome = etNome.text.toString()  
  
    if (nome.isBlank()) {  
        Toast.makeText(this, "Por favor, preencha o nome", Toast.LENGTH_SHORT).show()  
    } else {  
        Toast.makeText(this, "Salvo com sucesso!", Toast.LENGTH_SHORT).show()  
    }  
}
```

Exercício de Fixação

Crie uma `Activity` chamada `ConversorMoedaActivity`, que permita ao usuário converter valores entre **reais (R\$)** e **dólares (US\$)**, com interface visual utilizando bandeiras e botão de inversão.

A tela deve conter os seguintes elementos:

- **Bandeira do Brasil** (`ImageView`) do lado esquerdo
ID sugerido: `ivBandeiraOrigem`

- **Botão de inversão** com ícone de troca (↔) no centro
ID sugerido: `btnInverter`
- **Bandeira dos EUA** (`ImageView`) do lado direito
ID sugerido: `ivBandeiraDestino`
- **Campo de entrada numérica** (`EditText`) para digitar o valor a ser convertido
ID sugerido: `etValor`
- **Botão "Calcular"** (`Button`) para executar a conversão
ID sugerido: `btnCalcular`
- **Botão "Limpar"** (`Button`) para limpar o valor de `etValor`
ID sugerido: `btnCalcular`
- **Campo de resultado** (`TextView`) exibindo o valor convertido
ID sugerido: `tvResultado`

Separe a lógica em:

- `companion object`: embora o valor do dólar seja dinâmico, vamos usar uma constante com o valor atual do dólar, com o fim de treinar
- Atributos (`val`, `var` e `lateinit var`)
- `onCreate()`
- `setupViews()`
- `setupListeners()`
- Funções utilitárias como `converterMoeda()` e `inverterMoedas()`

Use validação com `Toast` se o campo estiver vazio ou inválido.

Use as imagens abaixo em xml (salve dentro de drawables):

bandeira_brasil.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="64dp"
    android:height="48dp"
    android:viewportWidth="64"
    android:viewportHeight="48">

    <path
        android:fillColor="#009739"
        android:pathData="M0,0h64v48h-64z" />
```

```

    <path
        android:fillColor="#FFCC29"
        android:pathData="M32,6L58,24 32,42 6,24z" />

    <path
        android:fillColor="#002776"
        android:pathData="M32,16a8,8 0 1,0 0,16a8,8 0 1,0 0,-16z" />
</vector>

```

bandeira_eua.xml

```

<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="64dp"
    android:height="48dp"
    android:viewportWidth="64"
    android:viewportHeight="48">

    <path android:fillColor="#B22234" android:pathData="M0,0h64v6h-64z" />
    <path android:fillColor="#B22234" android:pathData="M0,12h64v6h-64z" />
    <path android:fillColor="#B22234" android:pathData="M0,24h64v6h-64z" />
    <path android:fillColor="#B22234" android:pathData="M0,36h64v6h-64z" />

    <path android:fillColor="#FFFFFF" android:pathData="M0,6h64v6h-64z" />
    <path android:fillColor="#FFFFFF" android:pathData="M0,18h64v6h-64z" />
    <path android:fillColor="#FFFFFF" android:pathData="M0,30h64v6h-64z" />
    <path android:fillColor="#FFFFFF" android:pathData="M0,42h64v6h-64z" />

    <path android:fillColor="#3C3B6E" android:pathData="M0,0h25v24h-25z" />
</vector>

```

borda.xml

```

<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">

    <!-- Cor de fundo opcional -->
    <solid android:color="#FFFFFF" />

    <!-- Borda -->
    <stroke
        android:width="3dp"
        android:color="#000000" />

    <!-- Cantos arredondados opcionais -->
    <corners android:radius="8dp" />
</shape>

```

